



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

IPC Communication

The Noctalia plugin system allows plugins to register their own IPC handlers, making them accessible via the shell's IPC interface. This enables external scripts, keybindings, and other tools to interact with your plugin programmatically.

Overview

IPC (Inter-Process Communication) handlers allow your plugin to respond to commands from external sources. This is useful for:

- Triggering plugin actions from keybindings
- Integrating with external scripts and tools
- Automating plugin behavior
- Creating command-line interfaces for your plugin

When a plugin is loaded, its IPC handlers are automatically registered and can be called using the standard Noctalia IPC command format.

How It Works

Plugins register IPC handlers in their Main.qml file using the `IpcHandler` component from Quickshell. Each handler function becomes a callable command that can be invoked from the command line or other IPC clients.

The IPC system routes commands to the appropriate plugin handler based on the target prefix `plugin:your-plugin-id`.

Creating IPC Handlers

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

In your plugin's Main.qml file, register IPC handlers using the `IpcHandler` component:

```
import QtQuick
import Quickshell.Lo
import qs.Services.UI

Item {
    property var pluginApi: null

    IpcHandler {
        target: "plugin:your-plugin-id"
        function yourCommand() {
            // Your handler logic here
        }
    }
}
```

Target Naming

The target property must follow this format:

- **Prefix:** Always use "plugin:" as the prefix
- **Plugin ID:** Follow with your plugin's ID from manifest.json
- **Example:** If your plugin ID is "hello-world", use "plugin:hello-world"

Tip

The plugin ID in your target must exactly match the id field in your manifest.json file.

Plugin API Access

The `pluginApi` property is automatically injected by the plugin system and provides access to:

- **Settings:** pluginApi.pluginSettings - Read and write plugin settings
- **Persistence:** pluginApi.saveSettings() - Save settings to disk

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

- **Launcher:** pluginApi.openLauncher(screen) / pluginApi.closeLauncher(screen) / pluginApi.toggleLauncher(screen) - Control the launcher for provider plugins
- **Translations:** pluginApi.tr(key, interpolations) - Translate text
- **Metadata:** pluginApi.manifest - Access plugin manifest data

For complete API documentation, see the [Plugin API Reference](#).

Examples

Simple Command

A basic handler that updates a plugin setting:

```
import QtQuick
import Quickshell.Io
import qs.Services.UI

Item {
    property var pluginApi: null

    IpcHandler {
        target: "plugin:hello-world"
        function setMessage(message: string) {
            if (pluginApi && message) {
                pluginApi.pluginSettings.message = message;
                pluginApi.saveSettings();
                ToastService.showNotice("Message updated to: " + message);
            }
        }
    }
}
```

This handler can be called from the command line:

noctalia-shell:ipc@noctalia:~\$ cat Message "Hello from IPC!"

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Multiple Commands

You can define multiple functions within a single `IpcHandler`:

```
IpcHandler {  
  target: "plugin:my-plugin"  
  
  function toggle() {  
    if (!pluginApi) return;  
    pluginApi.pluginSettings.enabled = !pluginApi.pluginSettings.enabled;  
    pluginApi.saveSettings();  
  }  
  
  function enable() {  
    if (!pluginApi) return;  
    pluginApi.pluginSettings.enabled = true;  
    pluginApi.saveSettings();  
  }  
  
  function disable() {  
    if (!pluginApi) return;  
    pluginApi.pluginSettings.enabled = false;  
    pluginApi.saveSettings();  
  }  
  
  function setValue(value: string) {  
    if (!pluginApi) return;  
    var numValue = parseFloat(value);  
    if (!Number.isNaN(numValue)) {  
      pluginApi.pluginSettings.value = numValue;  
      pluginApi.saveSettings();  
    }  
  }  
}
```

Function Parameters

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

as strings and should be parsed as needed.

```
IpchHandler {  
  target: "plugin:my-plugin"  
  
  function setValue(value: string) {  
    // Parse string to number  
    var numValue = parseFloat(value);  
    if (Number.isNaN(numValue)) {  
      Logger.w("MyPlugin", "Invalid value:", value);  
      return;  
    }  
  
    pluginApi.pluginSettings.value = numValue;  
    pluginApi.saveSettings();  
  }  
  
  function setMultiple(param1: string, param2: string) {  
    // Handle multiple parameters  
    pluginApi.pluginSettings.param1 = param1;  
    pluginApi.pluginSettings.param2 = param2;  
    pluginApi.saveSettings();  
  }  
}
```

Caution

All IPC arguments are strings. Always parse numeric values, booleans, and other types as needed. Validate input before using it.

Calling IPC Commands

Once your plugin is loaded, you can call its IPC handlers using the standard Noctalia IPC command format:

noctalia-shell ipc call plugin:my-plugin setMessage "Hello from IPC!"

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Command Examples

Call a command with a string argument

```
qs -c noctalia-shell ipc call plugin:hello-world setMessage "Hello from IPC!"
```

Call a command with no arguments

```
qs -c noctalia-shell ipc call plugin:my-plugin toggle
```

Call a command with a numeric argument (passed as string)

```
qs -c noctalia-shell ipc call plugin:my-plugin setValue "42"
```

Integration with Keybindings

IPC commands can be bound to keyboard shortcuts in your Noctalia configuration:

```
{
  "keybinds": {
    "Super+Shift+P": "qs -c noctalia-shell ipc call plugin:my-plugin toggle"
  }
}
```

Best Practices

Input Validation

Always validate and parse input parameters before use:

```
function setValue(value: string) {  
  if (!pluginApi) {
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
    return;  
  }  
  
  var numValue = parseFloat(value);  
  if (Number.isNaN(numValue)) {  
    Logger.w("MyPlugin", "Invalid numeric value:", value);  
    ToastService.showError("Invalid value: " + value);  
    return;  
  }  
  
  pluginApi.pluginSettings.value = numValue;  
  pluginApi.saveSettings();  
}
```

Error Handling

Check for null/undefined values and handle errors gracefully:

```
function updateSetting(value: string) {  
  if (!pluginApi) {  
    Logger.e("MyPlugin", "Plugin API not available");  
    return;  
  }  
  
  if (!value || value.trim() === "") {  
    Logger.w("MyPlugin", "Value is required");  
    return;  
  }  
  
  // Proceed with update...  
  pluginApi.pluginSettings.setting = value;  
  pluginApi.saveSettings();  
}
```

User Feedback

Use the `updateSettings()` function to update settings.

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Services.UI

function updateSettings() {
  // ... update logic ...

  ToastService.showNotice("Settings updated successfully");
  // or
  ToastService.showError("Failed to update settings");
}
```

Logging

Use the `Logger` for debugging and error tracking:

```
import qs.Commons

function handleCommand(value: string) {
  Logger.d("MyPlugin", "Command called with value:", value);
  Logger.i("MyPlugin", "Settings updated");
  Logger.w("MyPlugin", "Warning: invalid input");
  Logger.e("MyPlugin", "Error occurred:", error);
}
```

Using Plugin API

Leverage the `pluginApi` for common operations:

```
// Save settings
pluginApi.saveSettings();
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Open/close panels - use withCurrentScreen for proper screen selection
pluginApi.withCurrentScreen(screen ⇒ {
  pluginApi.openPanel(screen);
});

// Or to close:
pluginApi.withCurrentScreen(screen ⇒ {
  pluginApi.closePanel(screen);
});

// Toggle launcher for provider plugins
pluginApi.withCurrentScreen(screen ⇒ {
  pluginApi.toggleLauncher(screen);
});

// Access translations
var text = pluginApi.tr("settings.enabled");
```

Tip

The `withCurrentScreen` helper is the preferred way to get a screen reference in IPC handlers. It automatically selects the appropriate screen based on cursor position.

Complete Example

Here's a complete example showing a plugin with multiple IPC handlers:

```
import QtQuick
import Quickshell
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons
import qs.Services.UI

Item {
    property var pluginApi: null

    lpcHandler {
        target: "plugin:example-plugin"

        // Toggle plugin state
        function toggle() {
            if (!pluginApi) return;

            pluginApi.pluginSettings.enabled = !pluginApi.pluginSettings.enabled;
            pluginApi.saveSettings();

            var status = pluginApi.pluginSettings.enabled ? "enabled" : "disabled";
            ToastService.showNotice("Plugin " + status);
            Logger.i("ExamplePlugin", "Toggled to:", status);
        }

        // Enable plugin
        function enable() {
            if (!pluginApi) return;

            pluginApi.pluginSettings.enabled = true;
            pluginApi.saveSettings();
            ToastService.showNotice("Plugin enabled");
        }

        // Disable plugin
        function disable() {
            if (!pluginApi) return;

            pluginApi.pluginSettings.enabled = false;
            pluginApi.saveSettings();
        }
    }
}
```

```
ToastService.showNotice("Plugin disabled");  
}
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
function setValue(value: string) {  
  if (!pluginApi) return;  
  
  var numValue = parseFloat(value);  
  if (Number.isNaN(numValue)) {  
    Logger.w("ExamplePlugin", "Invalid value:", value);  
    ToastService.showError("Invalid value: " + value);  
    return;  
  }  
  
  pluginApi.pluginSettings.value = numValue;  
  pluginApi.saveSettings();  
  ToastService.showNotice("Value set to: " + numValue);  
}  
  
// Open plugin panel  
function openPanel() {  
  if (!pluginApi) return;  
  
  pluginApi.withCurrentScreen(screen ⇒ {  
    pluginApi.openPanel(screen);  
  });  
}  
}  
}
```

Launcher Provider IPC

If your plugin provides a launcher provider, you can use the launcher API methods to toggle the launcher from IPC. This works correctly in both normal and overlay launcher modes.

```
import QtQuick
import Quickshell.io
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
Item {
    property var pluginApi: null

    lpcHandler {
        target: "plugin:my-provider"

        function toggle() {
            if (!pluginApi) return;
            pluginApi.withCurrentScreen(screen ⇒ {
                pluginApi.toggleLauncher(screen);
            });
        }
    }
}
```

The toggleLauncher method uses the commandPrefix from your manifest's metadata to determine the search prefix. For example, with "commandPrefix": "kaomoji", calling toggle will open the launcher with >kaomoji pre-filled.

```
# Toggle the launcher with your plugin's command prefix
qs -c noctalia-shell ipc call plugin:my-provider toggle
```



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
methods for each:
```

```
ipcHandler {
  target: "plugin:my-plugin"

  // Toggle the panel
  function togglePanel() {
    pluginApi.withCurrentScreen(screen => {
      pluginApi.togglePanel(screen);
    });
  }

  // Toggle the launcher with the provider's command prefix
  function toggle() {
    pluginApi.withCurrentScreen(screen => {
      pluginApi.toggleLauncher(screen);
    });
  }
}
```

Testing

After creating your IPC handlers, follow these steps to test them:

1. **Reload your plugin:** Disable and re-enable your plugin in Noctalia settings to ensure the handlers are registered
2. **Test via command line:** Use the IPC command format to call your handlers

```
qs -c noctalia-shell ipc call plugin:your-plugin-id yourCommand
```

3. **Check logs:** Monitor Quickshell logs for any errors or debug messages
4. **Verify behavior:** Ensure your plugin responds correctly to IPC calls

Troubleshooting

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

If your IPC handler isn't responding:

- **Verify target name:** Ensure it's exactly "plugin:your-plugin-id" (with the plugin: prefix)
- **Check plugin status:** Verify that your plugin is enabled and loaded
- **Function name:** Function names are case-sensitive - ensure exact matches
- **Review logs:** Check Quickshell/Noctalia logs for registration errors

Parameters Not Received

If parameters aren't being received correctly:

- **String conversion:** Remember that all IPC arguments are strings - parse them as needed
- **Parameter count:** Ensure the number of parameters matches your function signature
- **Type validation:** Always validate parameter types before use

Plugin API Not Available

If pluginApi is null or undefined:

- **Timing:** The pluginApi is injected after component creation - add null checks
- **Null checks:** Always verify pluginApi is available before using it
- **Component lifecycle:** IPC handlers remain active as long as the plugin is loaded

See Also

- [Getting Started](#) - Create your first plugin
- [Main Component](#) - Overview of Main.qml
- [Plugin API Reference](#) - Complete API documentation
- [Manifest Reference](#) - Plugin configuration

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)
